

Shiv Nadar University Chennai

Hemanth Kumar R

24011101035

B.Tech AI&DS 'A'

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 1

Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

3. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	Output Range	Typical Applications
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

4. Dataset Description

Dataset: Banknote Authentication Dataset

Source: UCI Machine Learning Repository

Download Link:

<https://archive.ics.uci.edu/dataset/267/banknote+authentication>

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

5. Experimental Procedure

Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

Expected Output

Dataset summary and statistical information.

Task 2: Exploratory Data Analysis

Generate

- Histograms
- Correlation Heatmap
- Scatter Plot
- Boxplots

Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

Task 4: Perceptron Implementation

Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

Task 5: Model Training

Display

- Epoch Number
- Misclassified Samples
- Updated Weights
- Updated Bias

Task 6: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

6. Source Code

The complete implementation of the Perceptron algorithm developed for this experiment is available in the GitHub repository below. The repository contains the Jupyter Notebook, source code, dataset, and all generated results.

Repository Link

<https://github.com/AI-MANIAC007/Hands-On-Deep-Learning/>

7. Experimental Results

The Banknote Authentication dataset was successfully loaded using the Pandas library. The dataset consists of four numerical input features (Variance, Skewness, Kurtosis, and Entropy) and one target variable (Class).

First Five Samples

The first five records of the dataset are shown in Table 7.1.

Table 7.1: First Five Samples of the Dataset

Index	Variance	Skewness	Kurtosis	Entropy	Class
0	3.62160	8.66610	-2.80730	-0.44699	0
1	4.54590	8.16740	-2.45860	-1.46210	0
2	3.86600	-2.63830	1.92420	0.10645	0
3	3.45660	9.52280	-4.01120	-3.59440	0
4	0.32924	-4.45520	4.57180	-0.98880	0

Dataset Dimensions

The dataset contains **1372 samples** and **5 columns**, as shown in Table 7.2.

Table 7.2: Dataset Dimensions

Rows	Columns
1372	5

Dataset Information

The dataset information is summarized in Table 7.3.

Table 7.3: Dataset Information

Feature	Non-Null Count	Data Type
Variance	1372	float64
Skewness	1372	float64
Kurtosis	1372	float64
Entropy	1372	float64
Class	1372	int64

Missing Value Analysis

The dataset was examined for missing values using the `isnull().sum()` function. The results are presented in Table 7.4.

Table 7.4: Missing Value Analysis

Feature	Missing Values
Variance	0
Skewness	0
Curtosis	0
Entropy	0
Class	0

No missing values were found in any feature of the dataset.

Descriptive Statistics

The statistical summary of the dataset is presented in Table 7.5.

Table 7.5: Descriptive Statistics of the Dataset

Statistic	Variance	Skewness	Curtosis	Entropy	Class
Count	1372.000	1372.000	1372.000	1372.000	1372.000
Mean	0.433735	1.922353	1.397627	-1.191657	0.444606
Standard Deviation	2.842763	5.869047	4.310030	2.101013	0.497103
Minimum	-7.042100	-13.773100	-5.286100	-8.548200	0.000000
25%	-1.773000	-1.708200	-1.574975	-2.413450	0.000000
50% (Median)	0.496180	2.319650	0.616630	-0.586650	0.000000
75%	2.821475	6.814625	3.179250	0.394810	1.000000
Maximum	6.824800	12.951600	17.927400	2.449500	1.000000

The dataset contains 1372 observations with four numerical input features and one binary target variable. No missing values were observed, and all features are numerical, making the dataset suitable for preprocessing and Perceptron-based binary classification.

8. Generated Plots with Interpretation

Feature Histograms

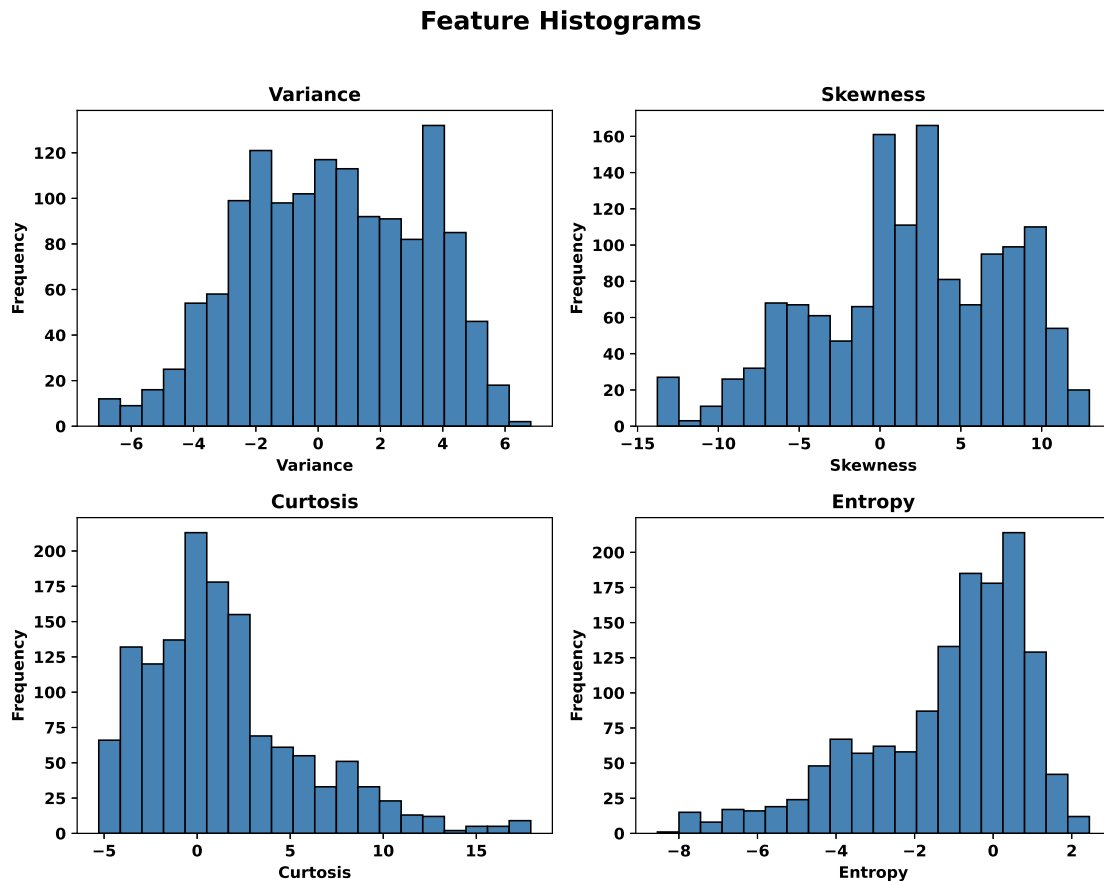


Figure 1: Feature Histograms of the Banknote Authentication Dataset

Variance and Skewness are negatively and positively skewed distributions that show a lot of diversification in textures and shapes of the data set. Kurtosis is positively skewed having a long tail on the right side. Entropy shows negative to almost zero value distributions with a few outliers. These distributions suggest that there are diverse values of features in the data set that will be helpful for the perceptron to learn how to classify banknotes.

Correlation Heatmap

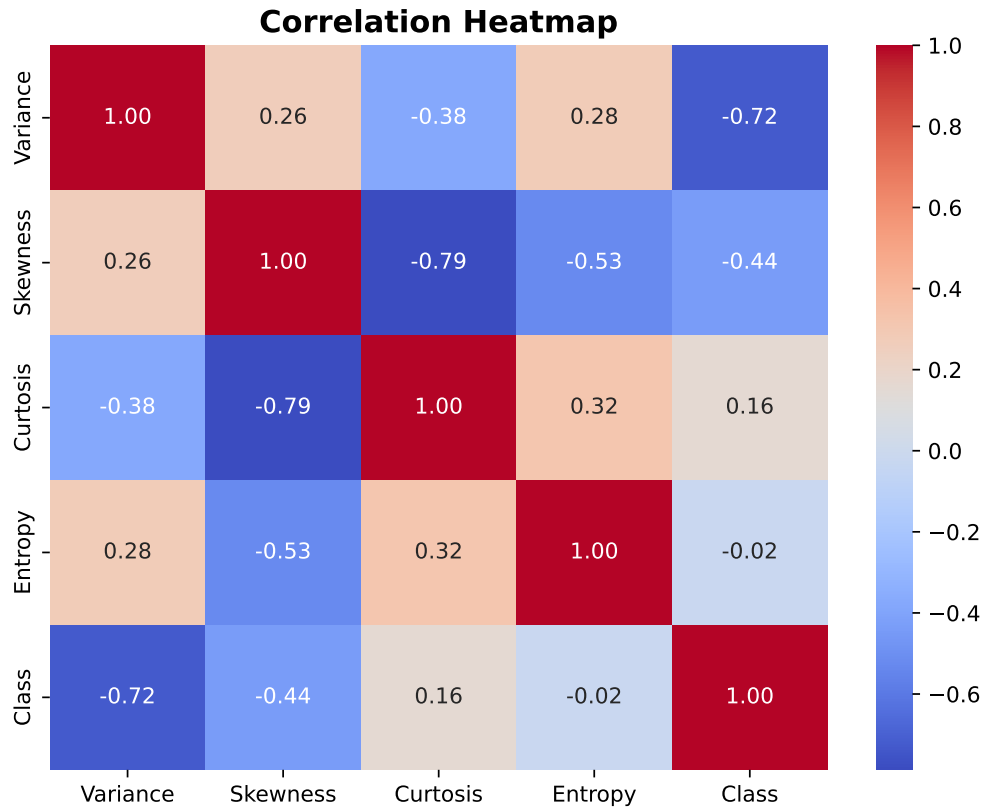


Figure 2: Correlation Heatmap of Dataset Features

From the correlation heatmap, the highly correlated features with the target class include Variance (-0.72), meaning that Variance is the most important variable that can be used to distinguish the real and fake bank notes. The Skewness and Kurtosis have high correlations (-0.79), meaning that they are collecting the same information. Entropy is highly uncorrelated with the class (-0.02). Conclusion: From the above, the presence of highly correlated and uncorrelated variables means that all the variables have their importance in the classification process.

Scatter Plot Matrix

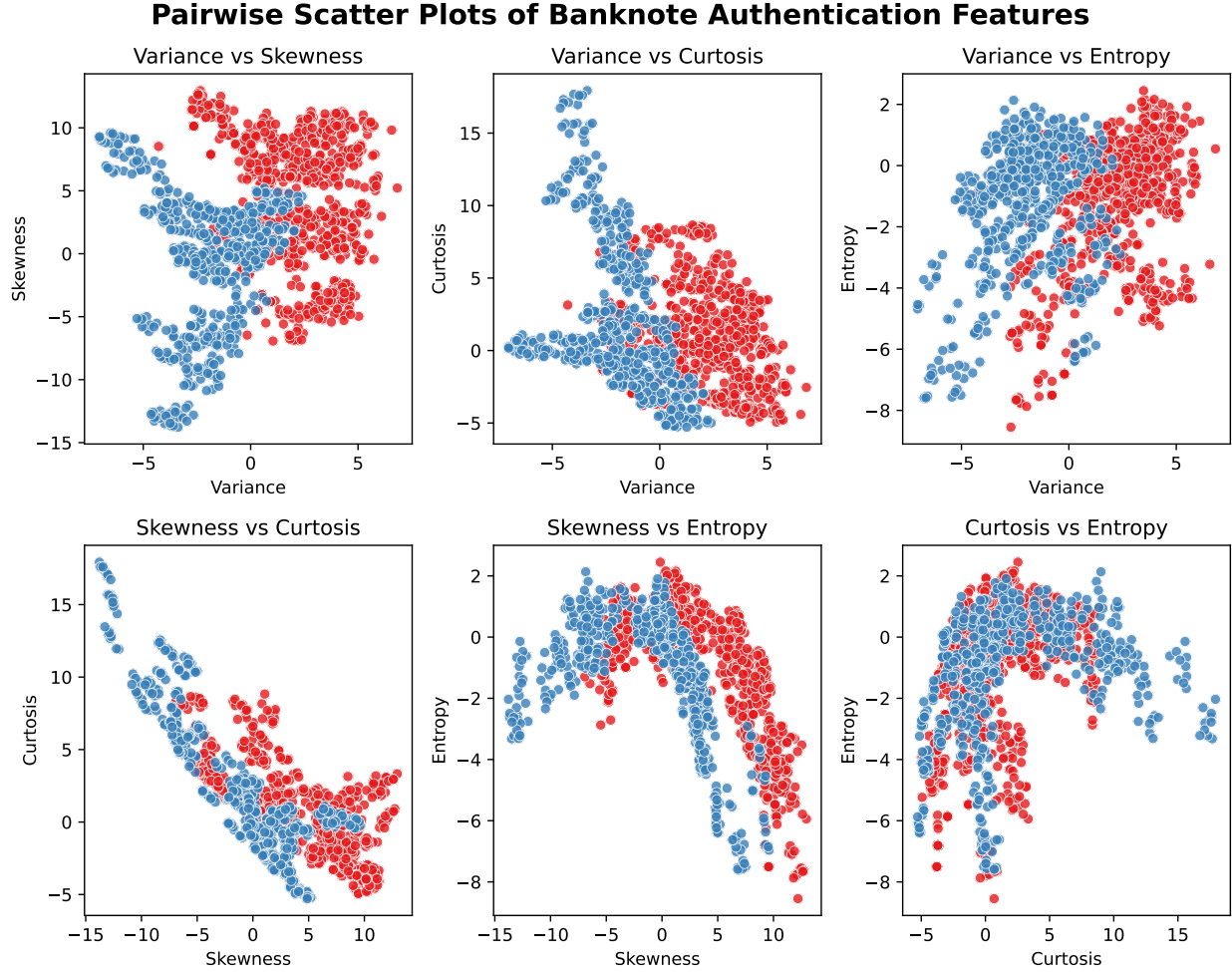


Figure 3: Scatter Plot Matrix Showing Pairwise Feature Relationships

Scatter plots of feature pairs help us understand the correlation and difference between the two classes. For example, the scatter plots of variance against skewness and variance against entropy clearly indicate class separation between the two classes, meaning that the feature selection is quite efficient in distinguishing between real and counterfeit bank notes. There are some pairs of features having non-linear correlation and overlapping between classes. It implies that only one feature cannot help distinguish between the classes perfectly; however, using multiple features will improve the classification performance.

Box Plots

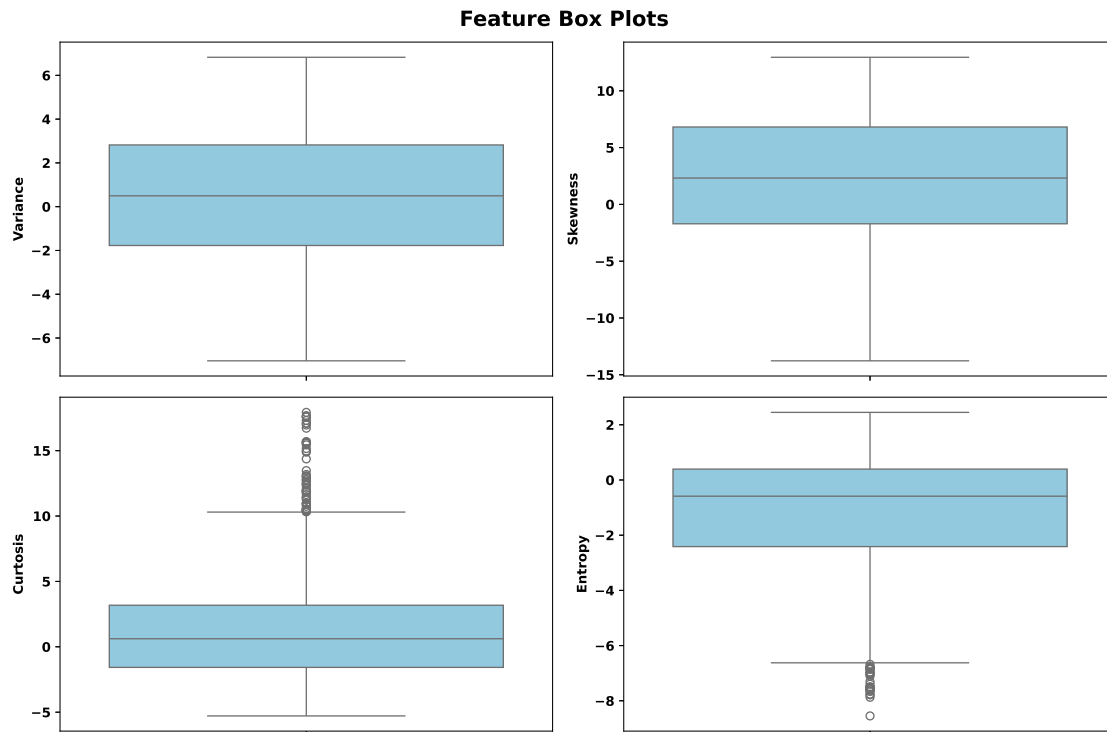


Figure 4: Box Plots of Numerical Features

The box plots summarise the spread and distribution of each numeric feature and whether outliers are present. Variance and Skewness have some balanced distribution in comparison with moderate variation while Kurtosis has some high value outliers and Entropy has some extreme low value observations. In general, the features are in different ranges and variability, which means that each feature provides different information to the dataset.

Training Error versus Epoch



Figure 5: Training Error versus Epoch

The training error reduces rapidly at the beginning stages of learning, and the error falls to about 20 from roughly 170 misclassified examples, suggesting that the Perceptron learns the hidden patterns in the dataset rather quickly. Once this stage is over, the error remains constant with small variations only.

Weight Evolution

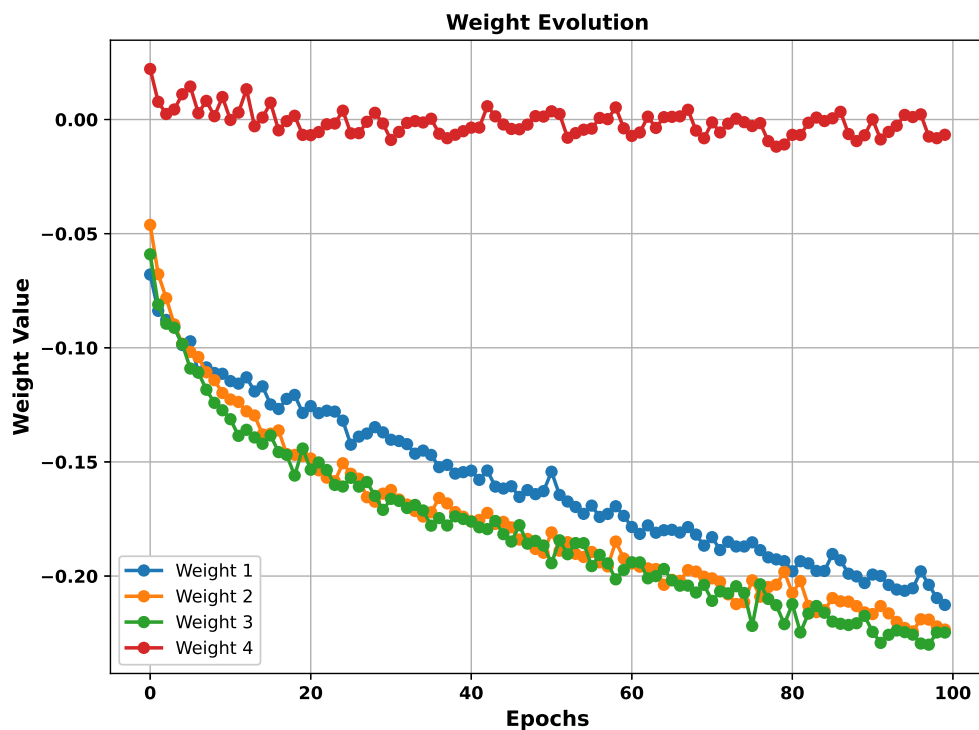


Figure 6: Evolution of Weights During Training

In the weight convergence plot, we observe the process of weight adjustments by the Perceptron in order to achieve maximum performance. We can see that Weights 1, 2, and 3 continuously decline until they reach a stable state, whereas Weight 4 remains at a level close to zero with minor variations. Therefore, it has less impact on the formation of the ultimate decision boundary.

Bias Evolution

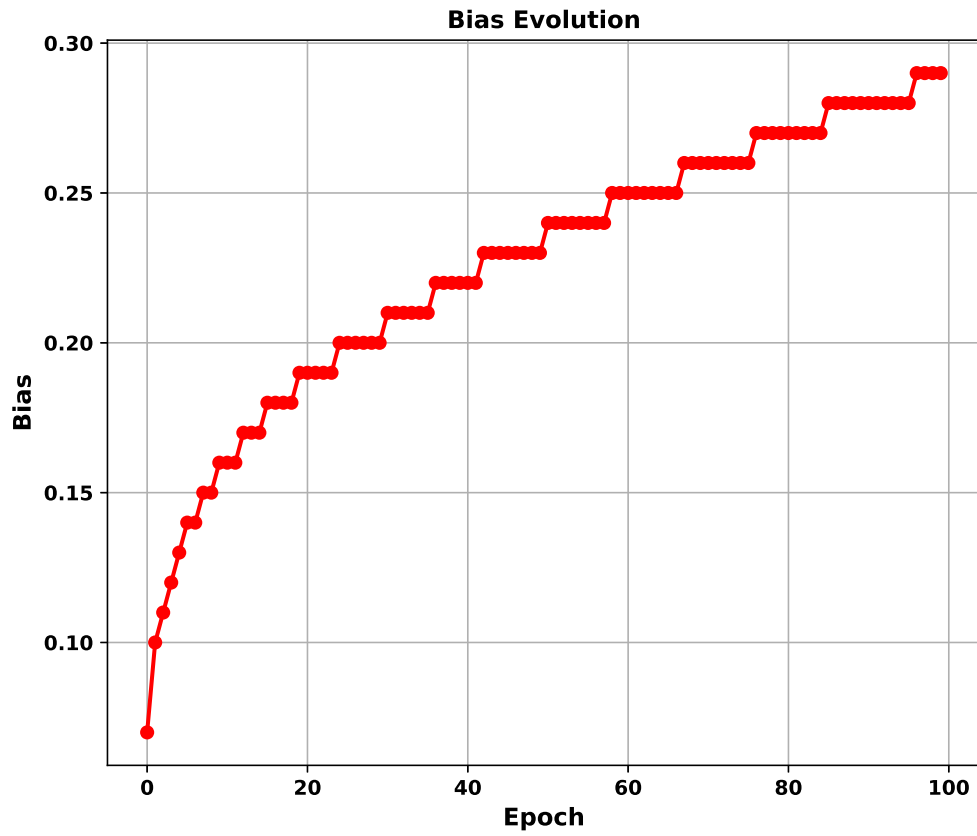


Figure 7: Evolution of Bias During Training

The graph of bias evolution reveals that the value of bias rises quickly at the beginning of training and then increases at a slower rate until it reaches a stable value of 0.29. This is an indication that the Perceptron is continually adjusting its threshold for error reduction.

Learning Rate Comparison

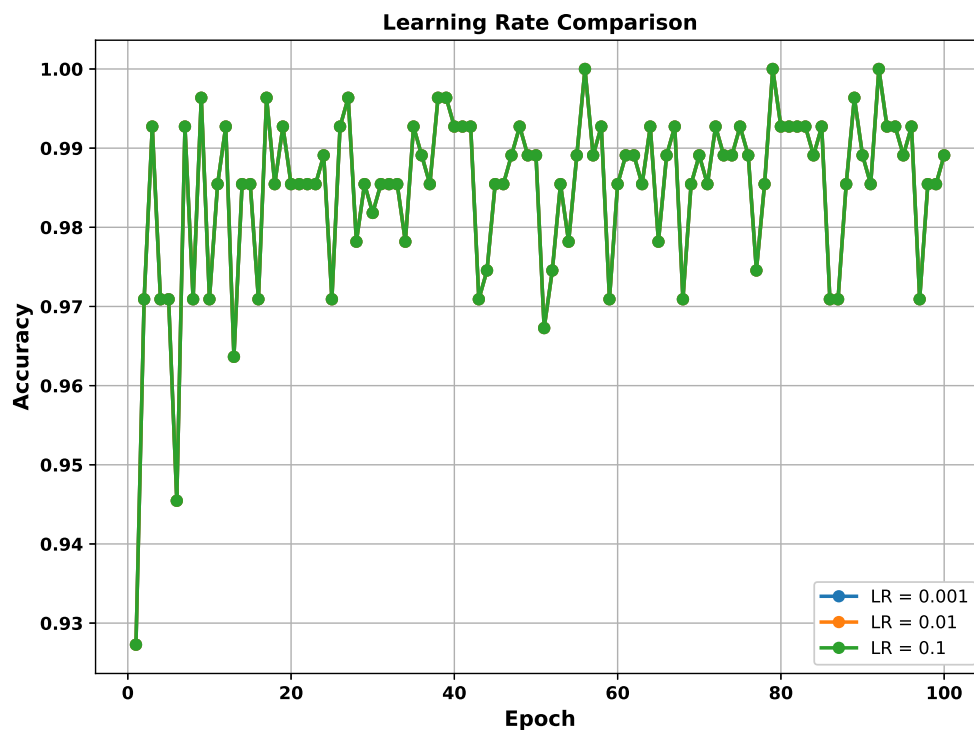


Figure 8: Comparison of Model Performance for Different Learning Rates

The results show that all three learning rates give almost equal levels of accuracy in training, with the network giving an accuracy rate of 98–99%. Although the high learning rate displays greater variance during the initial epochs, all three eventually achieve equal performance levels at the end of training.

Confusion Matrix

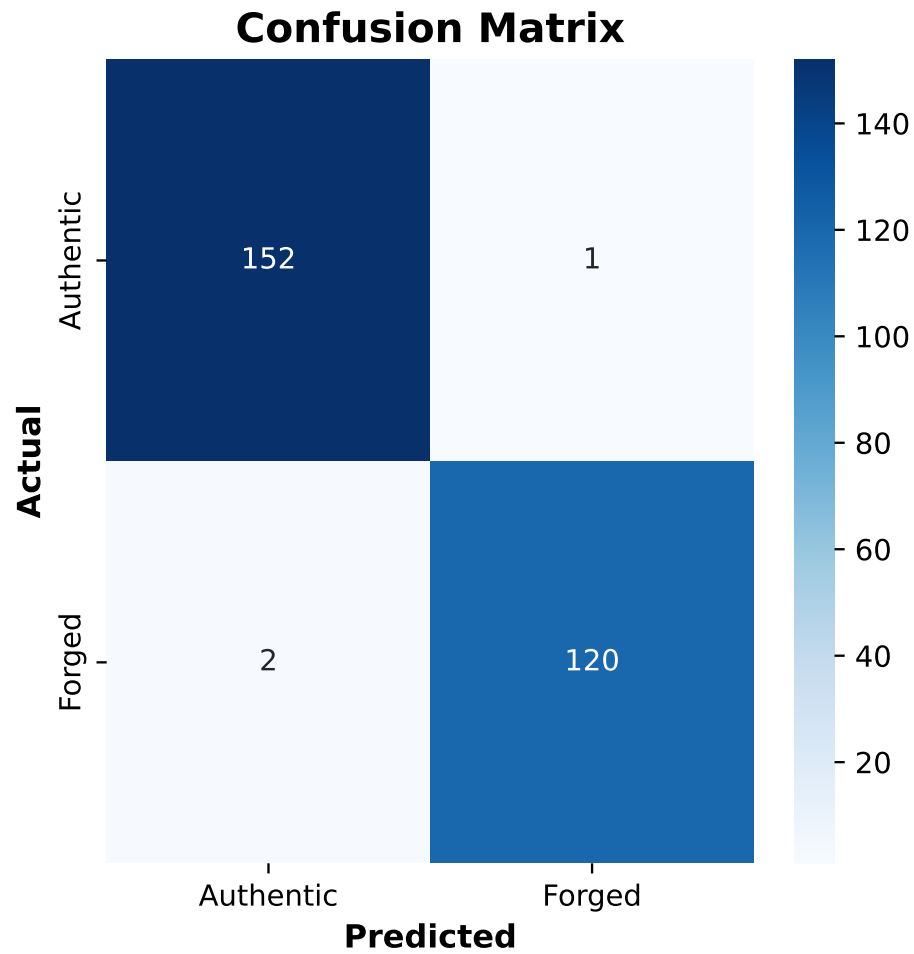


Figure 9: Confusion Matrix of the Perceptron Classifier

It can be seen from the confusion matrix that the perceptron model was able to classify 152 genuine banknotes and 120 fake banknotes, while wrongly classifying 1 genuine banknote as fake and 2 fake banknotes as genuine. The very few errors clearly indicate that the model is extremely good at separating the two classes.

Decision Boundary

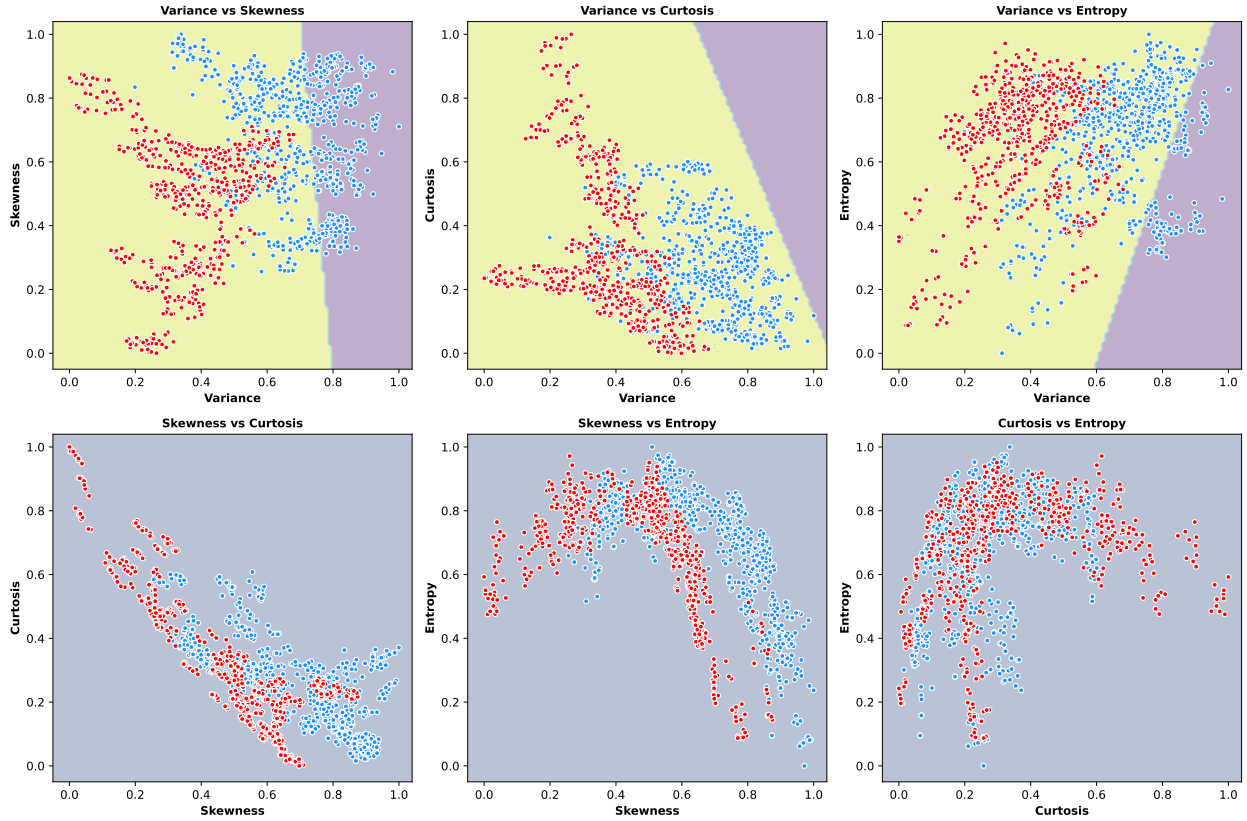


Figure 10: Decision Boundaries for Different Feature Pairs

The decision boundary graphs show how the Perceptron classifies the data points based on the different pair of features. Linear separability can be seen in the graphs with feature pairs such as Variance vs. Skewness, Variance vs. Kurtosis, and Variance vs. Entropy, which are able to clearly distinguish the authentic banknotes from the forged banknotes. There is very little overlapping between the two classes in these cases. On the other hand, there is heavy overlapping between the two classes in case of Skewness vs. Kurtosis, Skewness vs. Entropy, and Kurtosis vs. Entropy feature pairs, which makes them less suitable for linear separability.

Additional Tasks

Comparison of Step and Sigmoid Activation Functions

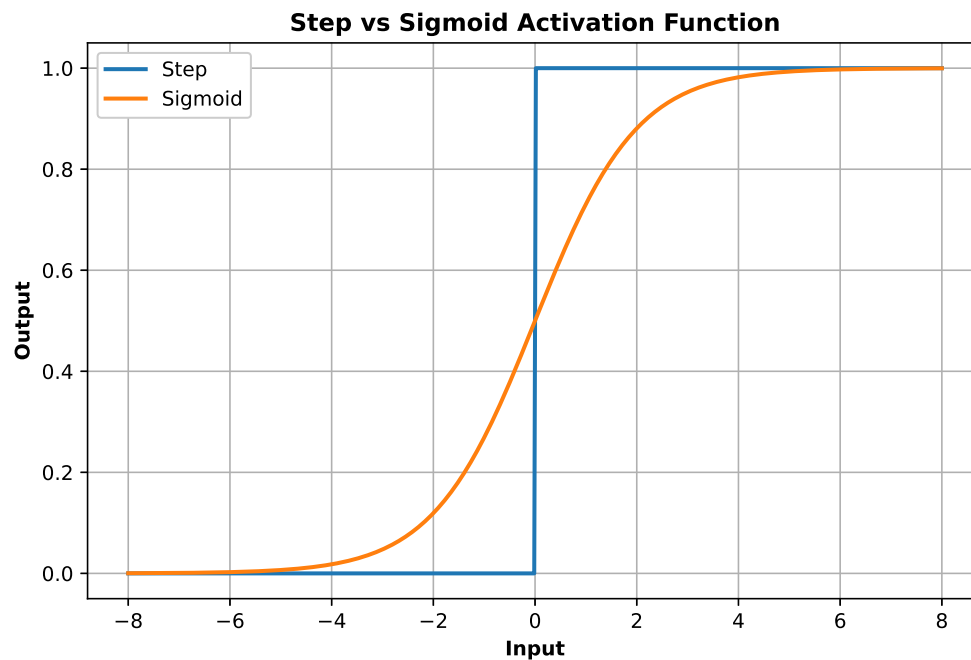


Figure 11: Comparison of Step and Sigmoid Activation Functions

Output of Step activation function is limited to just two values that are either 0 or 1, making it useful for classification using simple Perceptron model. On the other hand, output from Sigmoid activation function takes smooth values within the range of 0 to 1 and can therefore be considered as probabilities. This is depicted in the graph above where step function shows an abrupt change while sigmoid shows a gradual transition.

Step activation function is not used in modern deep learning because it is non-differentiable and has zero gradients which make it impossible to adjust weights using backpropagation algorithm. Sigmoid activation function on the other hand is differentiable and can therefore allow gradient-based optimization.

Comparison with Scikit-learn's Perceptron

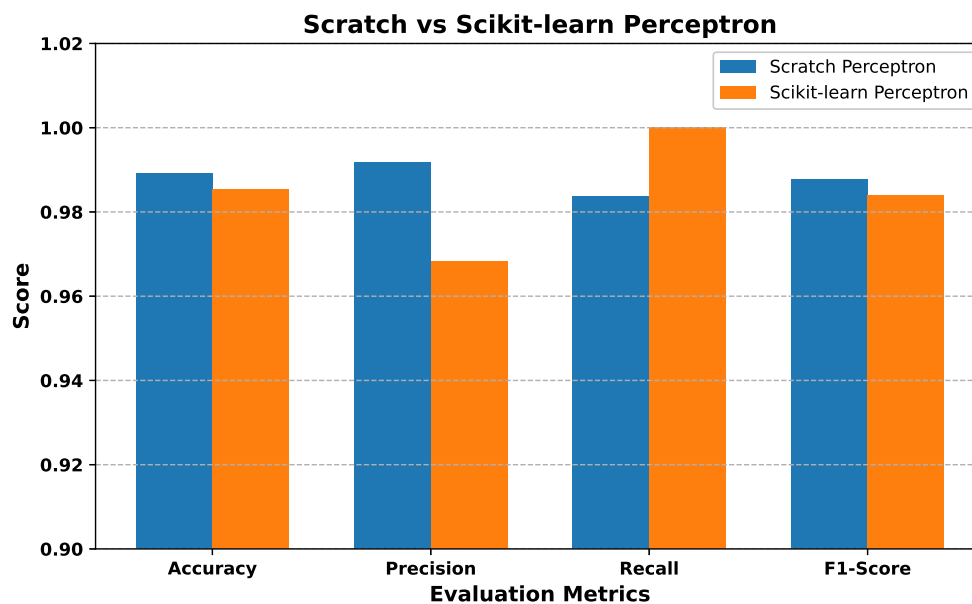


Figure 12: Performance Comparison between Scratch Implementation and Scikit-learn Perceptron

Performance comparison reveals that both scratch implementation and Scikit-learn Perceptron model perform almost equally on all measures. Scratch implementation is better than the latter when it comes to accuracy, precision, and F1-score. On the other hand, Scikit-learn Perceptron scores higher on the recall measure. Overall, the small difference between the two models indicates that the Perceptron model from scratch effectively mimics the learning process of Scikit-learn Perceptron implementation.

XOR Problem

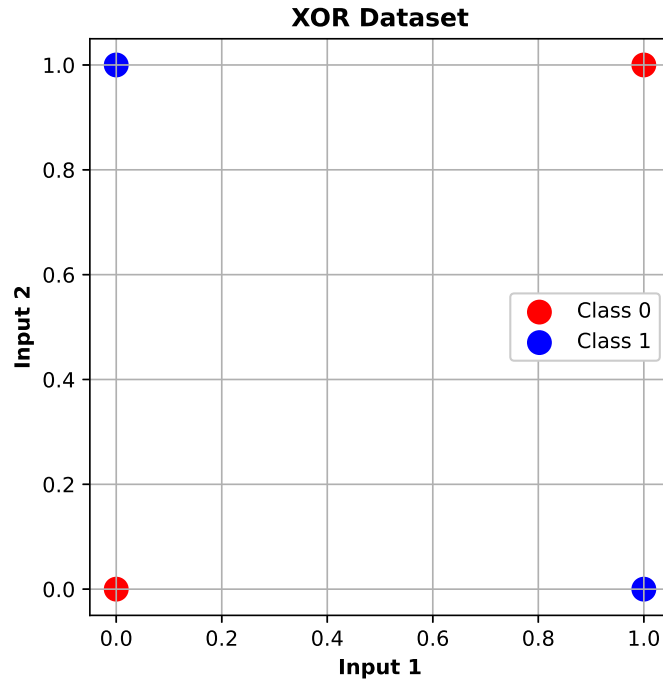


Figure 13: Visualization of the XOR Problem

In XOR dataset, the two classes lie diagonally opposite to one another, thus not allowing them to be linearly separated. In this case, there cannot exist any linear decision boundary which will separate all four data points into their respective classes. Since single layer Perceptron is capable of learning only linear decision boundaries, it is unable to solve the XOR problem. This is the reason why multi-layer neural networks were developed.

Effect of Feature Normalization on Model Convergence

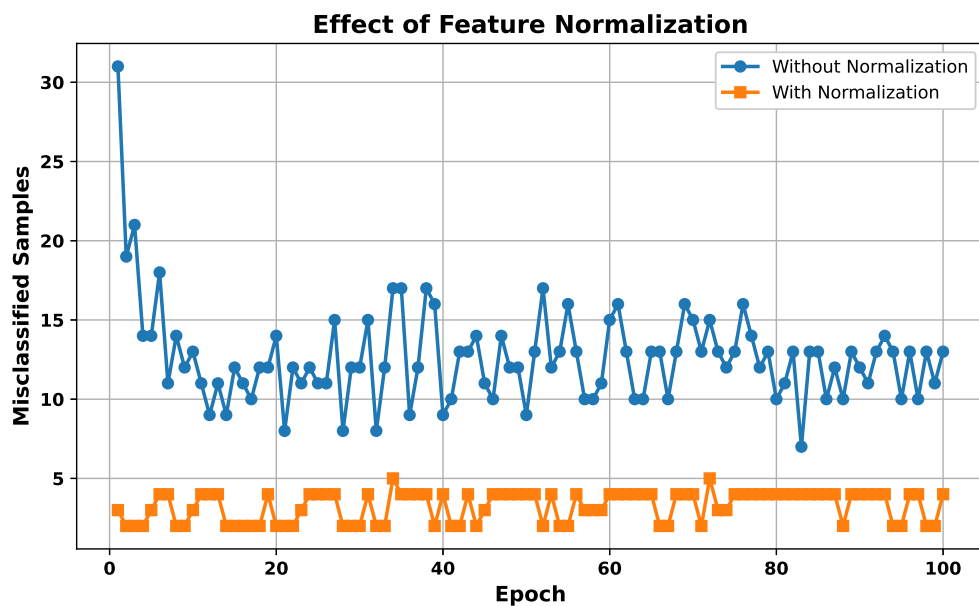


Figure 14: Effect of Feature Normalization on Perceptron Convergence

The plot clearly indicates that the Perceptron, which used normalized features for training, is faster to converge and always has fewer numbers of incorrectly classified data points. On the other hand, the perceptron that did not use normalized features has higher error rates and high variation levels throughout the entire training process. Overall, using normalized features for training improves the speed of convergence and the stability of the training process considerably.

Perceptron Learning for Logic Gates (AND, OR and NOT)

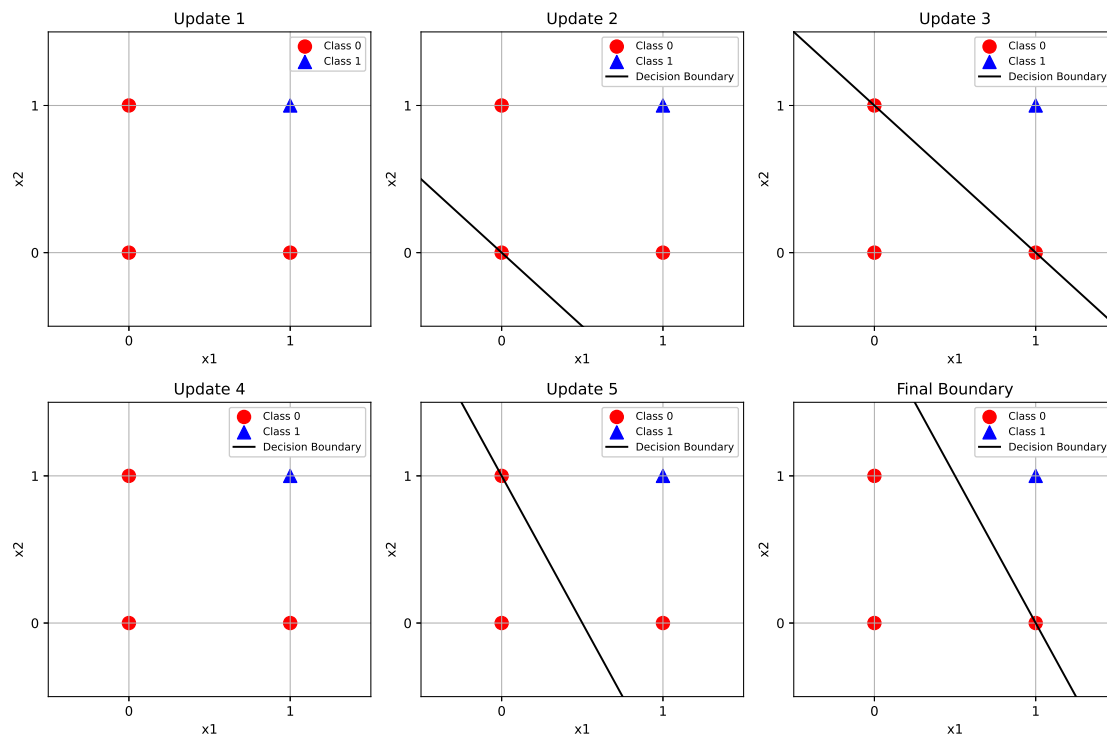


Figure 15: Decision Boundary Updates for the AND Gate

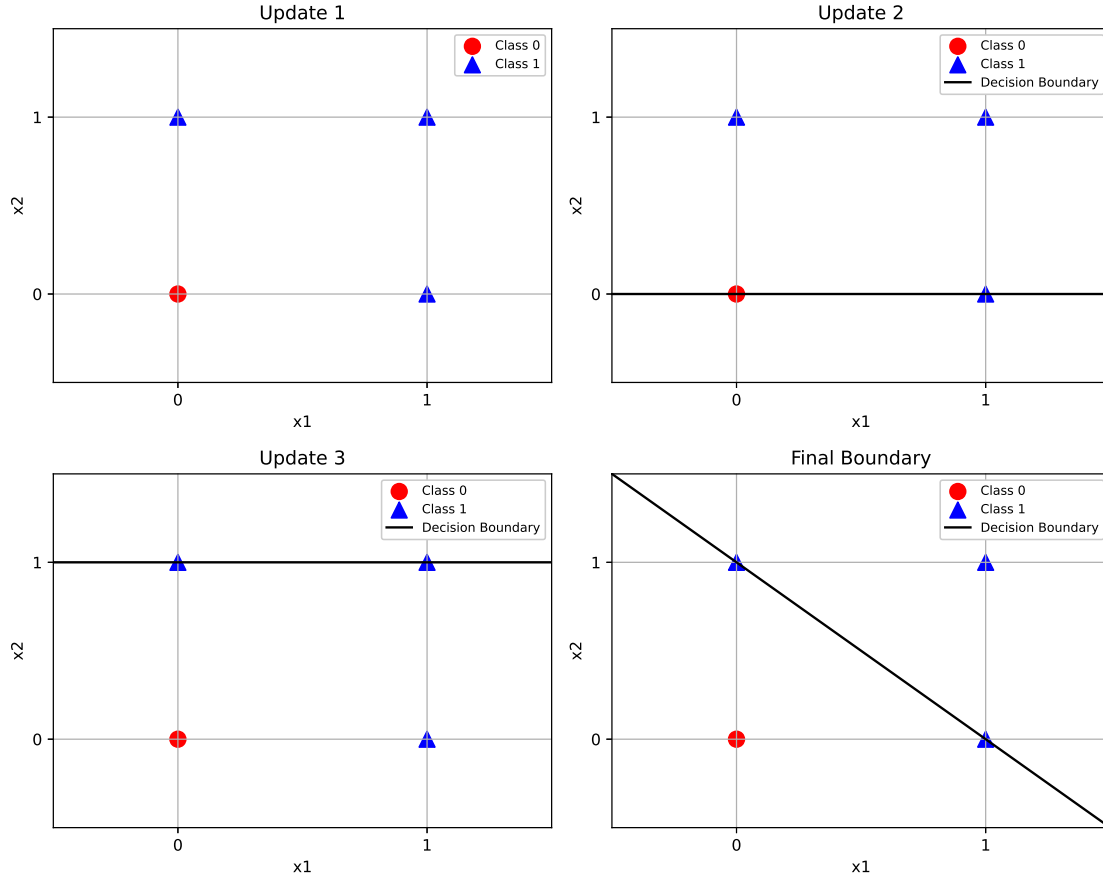


Figure 16: Decision Boundary Updates for the OR Gate

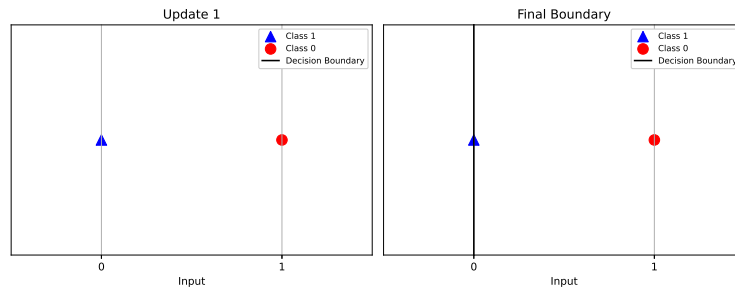


Figure 17: Decision Boundary Updates for the NOT Gate

The Perceptron Learning Algorithm was implemented for the AND, OR and NOT logic gates by updating the weights whenever a sample was misclassified. After every weight update, the corresponding decision boundary was plotted to observe how the model gradually learned the correct classification. Initially, the decision boundary was placed randomly because of random weight initialization. As training progressed, each weight update shifted the boundary closer to the optimal position.

The OR gate converged quickly because most of its input combinations belong to the positive class, making it easier for the Perceptron to identify a separating line. The AND gate required more updates since only one input combination belongs to the positive class, but it still converged

successfully because the data is linearly separable. The NOT gate converged almost immediately as it contains only a single input feature with two output classes that can easily be separated using a threshold.

Perceptron Learning for the XOR Gate

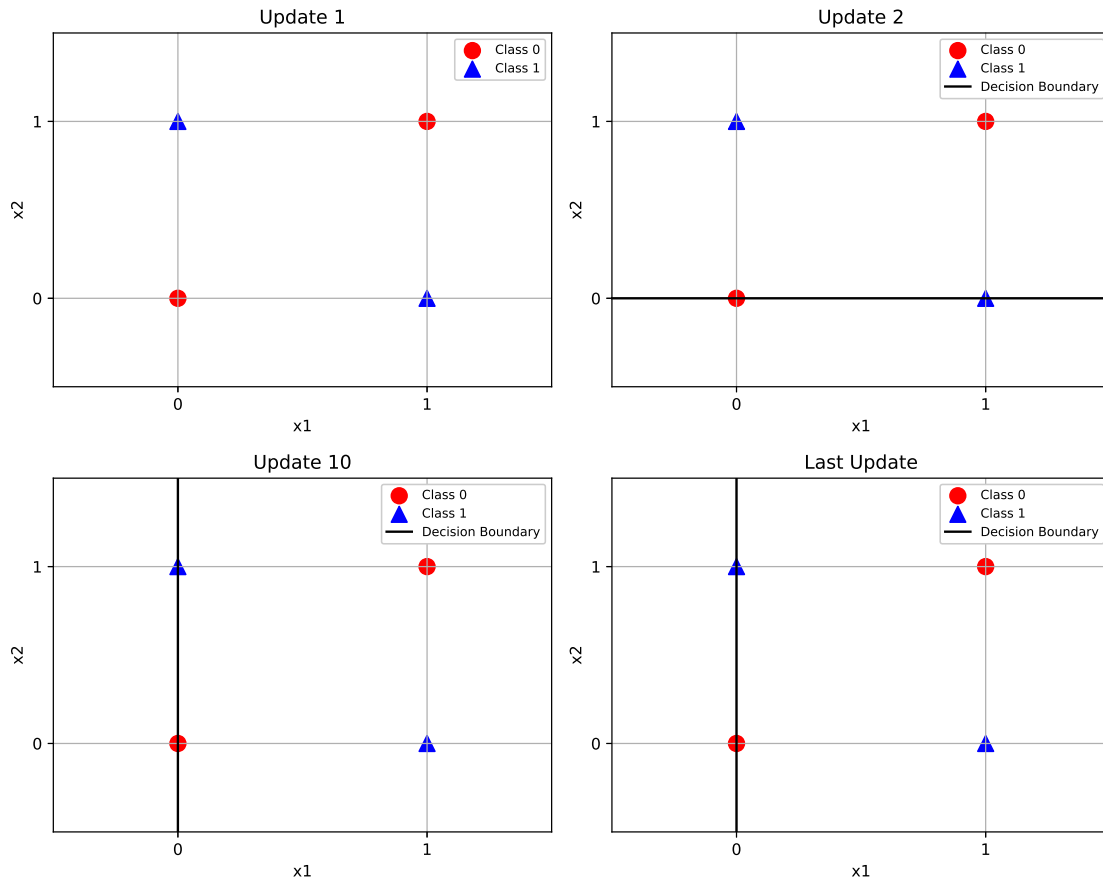


Figure 18: Decision Boundary Updates for the XOR Gate

The same Perceptron Learning Algorithm was used for the XOR gate with an updating of the weight after every mistake and drawing the decision boundary after every update. In contrast with the other logical gates AND, OR and NOT, where the algorithm converged after several updates of the weights, in the case of the XOR gate the algorithm never converged. This is due to the fact that weights never stopped changing because the model kept misclassifying some examples from the training set.

It happens because the XOR function is not linearly separable. Output classes of the XOR function are situated diagonally opposite to each other so there is no way to linearly separate them with one straight line. Anytime the Perceptron tries to do it, it makes mistakes for another pair of points.

9. Performance Tables

Training Summary

Dataset Size	1372 samples (1097 Training, 275 Testing)
Train/Test Split	80:20
Learning Rate	0.01
Maximum Epochs	100
Final Weights	[-0.0213, -0.0223, -0.0225, -0.0007]
Final Bias	0.0290
Accuracy	98.91%
Precision	99.17%
Recall	98.36%
F1-score	98.77%

Epoch-wise Learning

Epoch	Errors	Weight 1	Weight 2	Bias
1	171	-0.0679	-0.0462	0.07
2	101	-0.0838	-0.0677	0.10
3	43	-0.0878	-0.0783	0.11
4	43	-0.0908	-0.0898	0.12
5	57	-0.0988	-0.0983	0.13
6	43	-0.0972	-0.1018	0.14
7	30	-0.1108	-0.1041	0.14
8	49	-0.1086	-0.1107	0.15
9	28	-0.1110	-0.1142	0.15
10	43	-0.1114	-0.1198	0.16
11	24	-0.1146	-0.1227	0.16
12	18	-0.1157	-0.1237	0.16
13	33	-0.1130	-0.1278	0.17
14	30	-0.1191	-0.1296	0.17
15	24	-0.1169	-0.1380	0.17

10. Conclusion

This experiment gave a practical insight into how a Single-Layer Perceptron learns binary classification. By implementing the algorithm from scratch, we were able to better understand the role of

weights, bias, step activation function and Perceptron learning rule instead of treating the model as a black box. The reduction in training error, the evolution of weights and bias and the final decision boundary clearly indicated that the model was learning gradually from the misclassified samples.

The experimental results show that the Perceptron achieved high classification performance with very few misclassifications, which was also verified by the confusion matrix and the evaluation metrics. The study also discovered that feature normalisation is useful for convergence, making the learning process faster and more stable. The custom implementation was validated with the Scikit-learn Perceptron . Both models produced almost identical performance which confirmed the correctness of the implementation. Finally, in the analysis of the XOR problem, we pointed out an important limitation of the Single-Layer Perceptron: it can only learn linearly separable patterns. This experiment did not only help me to better understand the Perceptron algorithm, but also to understand that why modern neural networks have multiple layers and differentiable activation functions to solve more complex learning problems.

11. References

1. F. Rosenblatt, “The Perceptron,” *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.